

Computer Vision Object Detection & Classification System

Detailed Phased Development Plan — React + FastAPI + Vercel + Railway

Project Objective

Build a full-stack AI computer vision application that detects and classifies objects in images and later video/webcam input. The system will combine a deep-learning computer vision model with a production-style FastAPI backend and React.js frontend. The final application will be deployed with the frontend on Vercel and the backend/API on Railway.

Final Architecture

- User → React.js frontend → FastAPI API → Computer Vision model → predictions → React visualization.
- Frontend deployment: Vercel.
- Backend/API deployment: Railway.
- Model inference runs through the FastAPI backend.
- GitHub repository will contain clean frontend, backend, ML, configuration, testing, and documentation.

What We Will Develop

- Object detection — locate objects with bounding boxes and identify their classes.
- Image classification — classify images/categories where appropriate to the chosen project scope.
- Dataset pipeline — acquisition, validation, preprocessing, augmentation, and train/validation/test splits.
- Deep learning model — fine-tune a modern pretrained vision model.
- Evaluation — IoU, precision, recall, mAP and relevant classification metrics.
- Inference engine — process uploaded images and generate structured predictions.
- FastAPI backend — production-style inference API between the React frontend and ML model.
- React.js frontend — image upload, prediction display, bounding boxes, confidence scores, and results UI.
- Video/real-time extension — add video and optional webcam detection after the core system is stable.
- Deployment — React frontend on Vercel and FastAPI backend on Railway.
- Portfolio packaging — architecture diagram, screenshots, metrics, README, demo, and LinkedIn-ready summary.

Development Phases

Phase 1 — Project Foundation & Technical Design

- Define exact problem, use case, and target users
- Choose initial dataset and object classes
- Select detection/classification approach
- Design final React + FastAPI + ML architecture
- Create professional monorepo/repository structure
- Set up Python environment and React development environment
- Configure Git/GitHub workflow
- Define success metrics and acceptance criteria

Phase 2 — Dataset Acquisition & Exploration

- Acquire and organize the dataset
- Inspect class distribution and image quality
- Validate annotations
- Identify corrupted, duplicate, or problematic samples
- Create train/validation/test splits
- Build dataset visualization tools
- Document source, license, and limitations

Phase 3 — Data Preprocessing & Augmentation

- Implement preprocessing pipeline
- Implement appropriate image augmentation
- Resize/normalize according to the selected model
- Validate transformed samples
- Prevent data leakage
- Create reproducible data configuration

Phase 4 — Baseline Model & Training Pipeline

- Select pretrained architecture
- Configure training
- Implement training/validation pipeline
- Run first baseline experiment
- Save checkpoints and logs
- Record hyperparameters and environment details

Phase 5 — Model Evaluation & Error Analysis

- Calculate IoU, precision, recall and mAP
- Evaluate classification metrics where applicable
- Visualize predictions and failures
- Analyze false positives and false negatives
- Create baseline results table

Phase 6 — Model Improvement & Optimization

- Tune relevant hyperparameters
- Improve weak classes and difficult examples
- Compare experiments systematically
- Select the best model
- Measure inference latency and resource usage
- Prepare optimized inference model

Phase 7 — Production-Style Inference Engine

- Build reusable Python inference module
- Support image inference
- Return class, confidence, and bounding-box data
- Generate annotated outputs
- Add configurable confidence/IOU thresholds
- Add input validation and error handling

Phase 8 — FastAPI Backend

- Create FastAPI project structure
- Build image-upload inference endpoint
- Connect FastAPI to the ML inference engine
- Return clean JSON prediction responses
- Add CORS configuration for the React frontend
- Add API validation, errors, and health-check endpoint
- Test API locally

Phase 9 — React.js Frontend

- Create React.js + Vite application
- Build professional upload interface
- Connect frontend to FastAPI
- Display detected objects and confidence scores
- Render bounding boxes over images
- Add loading, error, and empty states
- Make the UI responsive and portfolio-quality

Phase 10 — Video / Real-Time Detection

- Add video-file inference
- Process video frame by frame
- Optionally add webcam/live detection
- Measure FPS and latency
- Optimize inference where necessary

Phase 11 — Testing, Deployment & Production Setup

- Add backend and frontend tests
- Verify API/frontend integration

- Prepare production environment variables
- Deploy React frontend to Vercel
- Deploy FastAPI backend to Railway
- Configure frontend-to-backend production URL
- Verify end-to-end production inference

Phase 12 — Portfolio Packaging & LinkedIn Release

- Create architecture diagram
- Add screenshots/GIFs and demo examples
- Document dataset, model, training, metrics and limitations
- Polish GitHub README
- Prepare final project description
- Prepare LinkedIn post and technical highlights
- Create stable GitHub release/version

Recommended Technology Stack

Layer	Technology
Frontend	React.js + Vite + Tailwind CSS
Frontend Hosting	Vercel
Backend/API	Python + FastAPI
Backend Hosting	Railway
Computer Vision	OpenCV
Deep Learning	PyTorch
Model	Modern pretrained object-detection architecture
Data / Analysis	NumPy, Pandas where useful
Visualization	Matplotlib
Testing	Pytest + frontend testing as appropriate
Version Control	Git + GitHub

Execution Rule

We will develop this project sequentially, one phase at a time. For each phase, you will receive the exact objectives, implementation tasks, files, commands, and expected outputs. After you finish the phase, you can share your code/results/errors. I will review it against the acceptance criteria. Only after the phase is confirmed complete will we move to the next phase.

Phase 1 Acceptance Criteria

- Problem statement and exact project scope are defined.
- Initial object classes and dataset direction are selected.
- React + FastAPI + ML architecture is documented.
- Professional repository structure is created.
- React and Python environments are configured.
- Initial README and Git workflow are established.
- No model training starts until Phase 1 is confirmed complete.